

WORKSHEET MAHASISWA

Modul 3 — Laravel Basic

Identitas	Isian
Nama	Sahizidan Lukman Hakim
NIM	251511060
Kelas	2B D3
Tanggal Mulai	21/09/2026
Tanggal Pengumpulan	
URL Repository GitHub	https://github.com/sahizidanlukman/MODUL3-6_PROYEK-3.git
Commit/Tag Modul 3 Proses	
Commit/Tag Modul 3 Final	

Petunjuk Penggunaan

Isi worksheet bersamaan dengan praktikum. Prediksi ditulis sebelum menjalankan kode. Bukti berupa tangkapan layar, output terminal, route:list, hasil Network, database, commit, atau SonarQube dicantumkan pada bagian yang sesuai. Seluruh source code disimpan pada repository GitHub; worksheet akhir dikumpulkan dalam format PDF.

- Jangan menyalin jawaban dari AI atau teman tanpa eksperimen dan verifikasi.
- Catat bantuan AI pada Log Bantuan dan Penggunaan AI.
- Pastikan URL repository dapat diakses dosen.

A. Aktivasi Pengetahuan Sebelumnya

A1. Menghubungkan Modul 1–2 dengan Laravel

Konsep Sebelumnya	Yang Sudah Saya Pahami	Hubungannya dengan Modul 3
HTML semantik	Struktur dasar pembuatan web yang berisikan tag yang memiliki makna	
CSS dan reusable component	Untuk mengatur tampilan halaman web dan memisahkan style agar dapat dipakai ulang	
JavaScript dan DOM	Javascript dipakai untuk membuat halaman jadi lebih interaktif, DOM Adalah representasi halaman HTML dalam bentuk struktur pohon yang ada di browser	
Event dan form	Event digunakan untuk menjalankan aksi seperti klik, ketik, submit, form yaitu elemen untuk menerima input pengguna	
Request/response di Network	Request yaitu permintaan yang dikirim browser ke server, response balasan dari server setelah menerima request	
Validasi input	Mengecek apakah input sudah benar sebelum dikirim	
Debugging dengan Console/Network	Melihat error untuk mencari tahu bagian yang bermasalah	

Jelaskan dengan kata-kata Anda sendiri: apa yang berubah ketika aplikasi mulai menggunakan Laravel di server, dan apa yang tetap dilakukan browser?

B. Peta Konsep dan Request Lifecycle

B1. Prediksi Alur Request

Sebelum menjalankan aplikasi

Tuliskan urutan yang Anda perkirakan terjadi ketika browser membuka GET /activities. Gunakan istilah browser, public/index.php, middleware, route, controller, Eloquent, Blade, dan response.

Browser mengirim request GET ke /activities, masuk lewat public/index.php, lalu diproses middleware. Router mencocokkan URL ke route di routes/web.php dan meneruskannya ke ActivityController@index. Controller mengambil data lewat Eloquent (Activity::get()), lalu mengirimnya ke Blade activities.index untuk disusun jadi HTML. HTML itu kemudian dikirim kembali sebagai response ke browser.

B2. Eksperimen 1 — Menelusuri Request Lifecycle

Tahap	Prediksi	Bukti Aktual	Penjelasan
Browser → route			
Route → controller			
Controller → Eloquent			
Controller → Blade			
Blade → response			

Bukti	Nama File / URL / Catatan
Network GET /activities	
php artisan route:list	
Log sementara controller	
File controller	
File Blade	
Kesimpulan request lifecycle dengan bahasa sendiri:	

C. Pertanyaan Pemahaman Fondasi Laravel

Apa perbedaan route dan controller?

Mengapa Blade tidak seharusnya melakukan query database?

Apa hubungan migration dengan struktur database?

Apa fungsi Eloquent model pada aplikasi?

Mengapa route model binding dapat mengurangi kode berulang?

Apa perbedaan response yang dibuat Laravel dengan DOM yang dibentuk browser?

Mengapa .env tidak boleh dikomit ke repository?

Kapan business rule perlu dipisahkan dari controller?

D. Task 1 — Laravel Skeleton dan Request Lifecycle

D1. Setup dan Baseline

Pemeriksaan	Hasil	Bukti
php -v		
composer --version		
php artisan --version		
Aplikasi dapat diakses		
Database dapat migrate		
Seeder berjalan		

D2. Struktur Implementasi

Komponen	Lokasi File	Tanggung Jawab yang Saya Pahami
Route		
ActivityController		
Activity model		
Migration		
ActivitySeeder		
activities/index Blade		
activities/show Blade		

D3. Skenario Uji Task 1

Skenario	Expected	Actual	Status	Bukti
Server awal	Aplikasi dapat diakses		☐ Lulus ☐ Gagal	
Daftar	Minimal lima kegiatan tampil		☐ Lulus ☐ Gagal	
Detail valid	Data yang dipilih tampil		☐ Lulus ☐ Gagal	
Detail tidak ada	HTTP 404		☐ Lulus ☐ Gagal	
Route map	Route menunjuk controller benar		☐ Lulus ☐ Gagal	
Data	Seeder dan field sesuai		☐ Lulus ☐ Gagal	

Masalah utama yang ditemukan pada Task 1 dan cara diagnosis nya:

E. Eksperimen 2 — Diagnosis Fat Controller

Instruksi

Klasifikasikan potongan kode sebelum melakukan refactor. Gunakan kategori: validation, business rule, persistence, response/presentation, atau orchestration.

Potongan Kode	Kategori	Lokasi yang Lebih Tepat	Alasan
\$request->validate(...)			
aturan Done → Planned			
validasi himpunan status			
\$activity->save()			
redirect + flash message			

Ciri yang membuat sebuah controller menjadi terlalu gemuk pada kasus ini:

F. Eksperimen 3 — Penempatan Business Rule

Pendekatan	Apa yang Terjadi	Risiko	Kesimpulan
Rule hanya pada Blade			
Rule diletakkan di service; Blade membantu UX			

Mengapa UI tidak boleh menjadi satu-satunya penjaga aturan aplikasi?

G. Task 2 — Activity CRUD dengan Form Request

G1. Resource Route dan Form Request

Komponen	Sudah?	Bukti / Lokasi
Resource route	☐ Ya ☐ Belum	
StoreActivityRequest	☐ Ya ☐ Belum	
UpdateActivityRequest	☐ Ya ☐ Belum	
Route model binding	☐ Ya ☐ Belum	
Create view	☐ Ya ☐ Belum	
Edit view	☐ Ya ☐ Belum	
Delete action	☐ Ya ☐ Belum	

G2. Business Rules BR-01 sampai BR-03

ID	Aturan	Lokasi Implementasi	Bukti
BR-01	Judul 5–100 karakter		
BR-02	Tanggal wajib dan valid		
BR-03	Status hanya Planned/Ongoing/Done		

G3. Skenario Uji CRUD

ID	Input / Aksi	Expected	Actual	Status
T2-01	Tambah valid	Satu data baru		☐ Lulus ☐ Gagal
T2-02	Title 4 karakter	Ditolak		☐ Lulus ☐ Gagal
T2-03	Status Unknown	Ditolak server		☐ Lulus ☐ Gagal
T2-04	Tanggal kosong	Ditolak		☐ Lulus ☐ Gagal
T2-05	Ubah title valid	Data berubah tanpa duplikasi		☐ Lulus ☐ Gagal
T2-06	Hapus satu data	Hanya target hilang		☐ Lulus ☐ Gagal
T2-07	Refresh setelah submit	Tidak menambah data		☐ Lulus ☐ Gagal
T2-08	Keyboard form	Kontrol dapat digunakan		☐ Lulus ☐ Gagal

Mengapa validasi sebaiknya berada pada Form Request, bukan diduplikasi di create dan edit Blade?

H. Task 3 — Business Logic Refactor

H1. Prediksi Transisi Status

Current	Next	Prediksi	Actual	Status
Planned	Planned	Diterima		
Planned	Ongoing	Diterima		
Ongoing	Ongoing	Diterima		

Current	Next	Prediksi	Actual	Status
Ongoing	Done	Diterima		
Done	Done	Diterima		
Done	Ongoing	Ditolak		
Done	Planned	Ditolak		
Ongoing	Planned	Ditolak		

H2. Pemetaan Tanggung Jawab Setelah Refactor

Aturan / Aktivitas	Controller	Form Request	Service	Model / Eloquent	Blade
Membaca input HTTP					
Validasi title 5–100					
Validasi status allowed					
Transisi status					
Menyimpan model					
Memilih response					
Menampilkan error					

Jelaskan mengapa service class digunakan pada kasus ini dan bukan sekadar karena “Laravel harus memakai service”:

I. Acceptance Criteria dan Bukti Pengujian

ID	Acceptance Criteria	Status	Bukti / Catatan
AC-01	Daftar kegiatan tampil.	☐ Lulus ☐ Gagal	
AC-02	Detail valid tampil; ID tidak ada menghasilkan 404.	☐ Lulus ☐ Gagal	
AC-03	Tambah valid menghasilkan satu data.	☐ Lulus ☐ Gagal	
AC-04	Judul <5 karakter ditolak.	☐ Lulus ☐ Gagal	
AC-05	Status di luar daftar ditolak.	☐ Lulus ☐ Gagal	
AC-06	Ubah valid tersimpan tanpa duplikasi.	☐ Lulus ☐ Gagal	
AC-07	Hapus hanya menghilangkan target.	☐ Lulus ☐ Gagal	
AC-08	Planned→Ongoing dan Ongoing→Done diterima.	☐ Lulus ☐ Gagal	
AC-09	Transisi mundur ditolak.	☐ Lulus ☐ Gagal	
AC-10	Alur normal tanpa error; static analysis tersedia.	☐ Lulus ☐ Gagal	

J. Eksperimen 4 — Static Analysis dengan SonarQube

Elemen	Baseline	Sesudah Refactor	Catatan
Tanggal scan			
Project key			
Code smell relevan			
Duplication / complexity yang diamati			
Maintainability issue			

J1. Satu Temuan yang Dipilih

Aspek	Isian Mahasiswa
Rule / issue	
File dan baris	
Penjelasan tool	
Mengapa relevan atau tidak relevan	
Perubahan yang dilakukan	
Acceptance criteria yang diuji ulang	
Hasil scan ulang	

Apakah semua temuan static analysis harus diperbaiki? Jelaskan berdasarkan satu temuan Anda.

K. Independent Challenge — Filter Kegiatan

Keputusan	Isian
Filter yang dipilih	
Query string	
Nilai valid	
Perilaku nilai tidak valid	
Lokasi query/filter logic	
Alasan penempatan	

K1. Acceptance Criteria Challenge

ID	Kriteria	Status	Bukti
F-01	Semua menampilkan seluruh data.	☐ Lulus ☐ Gagal	
F-02	Filter valid menampilkan subset tepat.	☐ Lulus ☐ Gagal	
F-03	Nilai filter tidak valid tidak menyebabkan error server.	☐ Lulus ☐ Gagal	
F-04	State filter dapat dibaca dari URL setelah reload.	☐ Lulus ☐ Gagal	

L. Homework — Architecture Review dan Refactor

Aspek	Sebelum	Perubahan	Alasan	Bukti Ulang
Bagian dipilih				
Masalah maintainability				
Refactor				
Acceptance criteria terkait				
Hasil Sonar / review				

Keputusan arsitektur yang paling penting pada homework dan alasannya:

M. Checkpoint Integrasi Modul 3

Definition of Done	Status	Bukti
Laravel 13 dan project dapat dijalankan dari repository.	☐	
Migration dan seeder berjalan.	☐	
CRUD normal case bekerja.	☐	
Form Request menegakkan BR-01–BR-03.	☐	
ActivityService menegakkan transisi status.	☐	
Route model binding digunakan.	☐	
Blade tidak melakukan query database.	☐	
AC-01–AC-10 diuji.	☐	
Laravel Pint dijalankan.	☐	
SonarQube baseline tersedia.	☐	
README lengkap.	☐	
Commit/tag modul-3-final tersedia.	☐	

N. Riwayat Commit / Checkpoint

Tahap	Commit / Tag	Tanggal	Catatan
Framework baseline			
Task 1			
Task 2			
Task 3			
Independent Challenge			
Homework			
Modul 3 Final			

O. Log Bantuan dan Penggunaan AI

Prinsip

Catat bantuan secara transparan. AI boleh membantu penjelasan konsep, membaca error, atau memberi hint setelah ada percobaan. Jawaban dan kode tetap harus diverifikasi melalui dokumentasi, request/response, database, route:list, atau eksperimen.

Tanggal / Fase	Pertanyaan / Sumber	Percobaan Sebelum Bantuan	Intisari Bantuan	Verifikasi / Perubahan Sendiri

Pilih satu saran AI/sumber. Jelaskan kembali konsepnya dengan bahasa sendiri dan bukti yang memverifikasinya:

P. Self Assessment

Pernyataan	1	2	3	4	5
Saya dapat menjelaskan request lifecycle Laravel.					
Saya membedakan route, controller, Form Request, service, model, migration, dan Blade.					
Saya dapat membuat migration dan memakai Eloquent.					
Saya dapat membuat resource CRUD dan route model binding.					
Saya membedakan validation rule dan business rule.					
Saya dapat menulis acceptance criteria.					
Saya dapat membaca temuan SonarQube secara kritis.					
Saya dapat menjelaskan dan mengubah kode tanpa AI.					

Q. Peer Assessment

Kriteria	Skor 1–5	Catatan yang Dapat Ditindaklanjuti
Alur request		
CRUD		
Business logic placement		
Clean code		
Repository / README		
Quality evidence		

R. Lesson Learnt

Konsep yang sekarang benar-benar saya pahami:

Kesalahan yang paling membantu saya belajar:

Perbedaan validation rule dan business rule menurut saya:

Keputusan arsitektur yang paling penting:

Temuan static analysis yang paling bermanfaat:

Bagian yang berhasil saya selesaikan sendiri:

Target kompetensi sebelum Modul 4:

S. Checklist Pengumpulan

Pemeriksaan	Status
Worksheet telah diisi seluruhnya.	☐
Bukti dapat dibaca.	☐
URL repository GitHub tercantum dan dapat diakses.	☐
README dapat digunakan untuk menjalankan project.	☐
Commit/tag modul-3-final tersedia.	☐
Worksheet diekspor sebagai PDF dengan nama yang ditetapkan dosen.	☐